

Capstone Deliverable 2: Architecture Diagram & Requirements

Project: Heirloom, non-custodial digital succession for Solana wallets

Team: Heirloom (Rohit Rathore, Josh Sandhu)

Editable diagrams: every diagram in this document is also supplied as an Excalidraw file (.excalidraw), named to match the figure numbers below.

1. Scope: capstone idea and MVP use cases

1.1 The idea

Heirloom lets a Solana owner name a beneficiary and a check-in interval. While the owner is alive, **all assets stay in the owner's own wallet**: no vault, no escrow, no shared keypair. If the owner stops checking in, a pre-authorised sweep can be completed and broadcast, moving the owner's live SOL balance to the beneficiary.

1.2 Core mechanism

- Each check-in cycle the owner signs a durable-nonce transaction `[AdvanceNonceAccount, heir_sweep(beneficiary)]`. `heir_sweep` has **no amount argument**: it reads the owner's live lamports at execution time (confirmed in our proof of concept).
- **Problem found in the POC**: anyone holding a fully signed copy can submit it early. The failed attempt still advances the nonce and permanently burns the only copy.
- **Adopted fix**: the Heirloom keeper is both the nonce authority and the fee payer, so the stored transaction is structurally incomplete (it lacks the keeper signature) until the deadline passes. Solana rejects a transaction with a missing signature before any instruction runs, so a premature replay by anyone else cannot advance or burn the nonce. The owner-signed partial transaction is therefore safe to store in the Plan PDA, unencrypted.

Who is the co-signer? A program has no private key. A Solana program (and a PDA) cannot sign a top-level transaction: transaction signatures are ed25519 signatures from a keypair, and a PDA is deliberately off-curve. A PDA can only "sign" inside a CPI made by its own program (`invoke_signed`), which cannot cover the fee payer or the nonce authority of the transaction itself. So the withheld co-signer is the **keeper keypair**, an ordinary off-chain key held by the Heirloom service (HSM / KMS post-MVP). The **program's contribution is the on-chain hard gate** inside `heir_sweep` (REQ08 to REQ12): even a fully signed transaction cannot move funds before the deadline.

The keeper cannot act early: two layers. (1) On-chain, REQ08 reverts any `heir_sweep` before `last_checkin + interval + grace`, so no funds can move early. (2) `AdvanceNonceAccount` must be instruction 0 and runs before our program, so a revert at instruction 1 still leaves the nonce advanced. The on-chain check therefore cannot protect the nonce, so the keeper has an explicit off-chain hard guard (REQ26): it re-reads the Plan from the chain at finalized commitment immediately before signing and refuses to sign unless the deadline plus a safety margin has passed, then simulates (REQ24).

Compromise of the keeper key is the residual risk and the reason HSM / KMS and an N-of-M signer set are on the roadmap.

1.3 MVP use cases (1 use case = 1 atomic state transition = 1 Anchor instruction handler)

ID	Use case	Anchor handler	Signer(s)	Atomic state transition
UC1	Owner creates a succession plan	<code>initialize_plan</code>	Owner	No account → Plan PDA (status Active, <code>last_checkin</code> = now, <code>sweep_tx</code> stored)
UC2	Owner proves liveness	<code>check_in</code>	Owner	<code>plan.last_checkin</code> = Clock now; <code>plan.sweep_tx</code> replaced
UC3	Sweep live SOL to the beneficiary after the deadline	<code>heir_sweep</code>	Owner (presigned earlier, offline) + Keeper (signature withheld until the deadline)	Owner lamports → beneficiary; <code>plan.status</code> = Distributed
UC4	Owner cancels the plan and removes the heir	<code>close_plan</code>	Owner	Plan PDA → closed (rent to owner, <code>sweep_tx</code> deleted)
UC5	Owner updates the check-in period	<code>update_plan</code>	Owner	<code>checkin_interval</code> / <code>grace_period</code> replaced, <code>last_checkin</code> = now, <code>sweep_tx</code> replaced

Out of MVP (roadmap): SPL-token distribution, multi-beneficiary splits, in-place beneficiary change (MVP: `close_plan` then `initialize_plan`), fixed amount with max failover, staged warning instructions, protocol pause / admin config, ZK / privacy, post-trigger market positions. The grace period is a plan field (`grace_period`) changed through `update_plan`, not a separate instruction. “Transfer to my own failsafe wallet” needs no extra instruction: it is `beneficiary = owner’s other wallet`.

1.4 Actors (LOI categories carried forward)

Category	Actor	Role in MVP	Signs
Direct	Owner	Creates, updates and cancels the plan, checks in, presigns the sweep each cycle	UC1, UC2, UC4, UC5, and (earlier, offline) the owner half of UC3
Direct	Keeper (Heirloom relay)	Reads the stored half-signed sweep from the Plan PDA, watches the deadline, adds the final signature and broadcasts; burns the nonce after a plan is closed	UC3 (co-signer, fee payer, nonce authority)
Beneficiary	Beneficiary	Receives SOL passively	Nothing
Administrator	(none on-chain in MVP)	ProtocolConfig / admin pause deferred to roadmap	—
Stakeholder	Wallet providers, estate / legal advisors, other legal heirs, auditors	Off-chain interest only	—

1.5 Suggested build milestones

- **Week 1:** Plan PDA, nonce account creation, `initialize_plan`, `check_in`.
- **Week 2:** `heir_sweep` with all constraints, keeper scan and pre-flight.
- **Week 3:** `close_plan`, `update_plan`, keeper nonce burn, tests and demo.

2. Atomic requirements and traceability

Every requirement is a single, testable action. Each carries a label on the diagrams (last column), so a reviewer can start from a use case and find where it happens, or start from a diagram label and find the requirement.

Req	Atomic statement	Anchor handler	UC	Diagram label
REQ01	The protocol shall create a Plan PDA with seeds ["plan", owner], signed by the owner, and fail if one already exists (one plan per owner).	<code>initialize_plan</code>	UC1	REQ01 on the "Plan PDA already exists?" decision (Flow 02) and on the "1 initialize_plan" arrow (Diagrams 01, 09)
REQ02	The protocol shall reject a plan whose <code>checkin_interval</code> or <code>grace_period</code> is zero or outside the allowed bounds.	<code>initialize_plan</code>	UC1	REQ02 on the "interval and grace within bounds?" decision (Flow 02)
REQ03	The protocol shall reject a plan whose nonce account authority is not the keeper.	<code>initialize_plan</code>	UC1	REQ03 on the "nonce authority == keeper?" decision (Flow 02)
REQ04	The protocol shall store the owner-signed partial sweep transaction in <code>plan.sweep_tx</code> and reject it if it exceeds the allocated space.	<code>initialize_plan</code>	UC1	REQ04 on the " <code>sweep_tx</code> length $\leq$ allocated space?" decision and the CREATE Plan PDA box (Flow 02)
REQ05	The protocol shall allow only <code>plan.owner</code> to call <code>check_in</code> , and only while <code>plan.status = Active</code> .	<code>check_in</code>	UC2	REQ05 on the two guard decisions (Flow 03)
REQ06	The protocol shall set <code>plan.last_checkin</code> to the Clock time on <code>check_in</code> .	<code>check_in</code>	UC2	REQ06 on the UPDATE Plan PDA box and the Clock input (Flow 03)
REQ07	The protocol shall overwrite <code>plan.sweep_tx</code> with the freshly signed partial sweep on <code>check_in</code> .	<code>check_in</code>	UC2	REQ07 on the <code>sweep_tx</code> size decision and UPDATE Plan PDA box (Flow 03)
REQ08	The protocol shall revert <code>heir_sweep</code> unless $\text{Clock} \geq \text{last_checkin} + \text{checkin_interval} + \text{grace_period}$.	<code>heir_sweep</code>	UC3	REQ08 on the " $\text{Clock} \geq \dots$?" decision, no branch to <code>DeadlineNotReached</code> (Flow 04)
REQ09	The protocol shall revert <code>heir_sweep</code> unless the beneficiary account equals <code>plan.beneficiary</code> and the owner account equals <code>plan.owner</code> .	<code>heir_sweep</code>	UC3	REQ09 on the binding decision (Flow 04)
REQ10	The protocol shall revert <code>heir_sweep</code> unless the keeper signer equals <code>plan.keeper</code> and the nonce account equals <code>plan.nonce_account</code> .	<code>heir_sweep</code>	UC3	REQ10 on the keeper/nonce decision (Flow 04)
REQ11	The protocol shall revert <code>heir_sweep</code> unless instruction 0 of the transaction is <code>AdvanceNonceAccount</code> on <code>plan.nonce_account</code> .	<code>heir_sweep</code>	UC3	REQ11 on the "ix 0 is <code>AdvanceNonce</code> ?" decision and <code>Instructions</code> sysvar input (Flow 04, Diagram 09)

Req	Atomic statement	Anchor handler	UC	Diagram label
REQ12	The protocol shall revert heir_sweep unless plan.status = Active.	heir_sweep	UC3	REQ12 on the "status == Active?" decision, no branch to AlreadyDistributed (Flow 04)
REQ13	The protocol shall transfer the owner wallet's live lamport balance to the beneficiary through a System Program transfer CPI (no amount argument).	heir_sweep	UC3	REQ13 on the "CPI system_program::transfer" arrow/box (Flows 04, Diagrams 01, 09)
REQ14	The protocol shall set plan.status = Distributed after the transfer.	heir_sweep	UC3	REQ14 on the "Plan.status = Distributed" box (Flow 04, Diagrams 01, 09)
REQ15	The protocol shall allow only plan.owner to close the Plan PDA (status Active or Distributed), returning rent to the owner and deleting sweep_tx.	close_plan	UC4	REQ15 on the guard decisions and the CLOSE Plan PDA box (Flow 05)
REQ16	Off-chain: the keeper shall advance and withdraw the nonce account (rent to owner) after it observes that the Plan PDA no longer exists.	none (keeper, off-chain)	UC4	REQ16 on the "AdvanceNonce + WithdrawNonce" box (Flows 05, 07)
REQ17	The protocol shall allow only plan.owner to call update_plan, and only while plan.status = Active.	update_plan	UC5	REQ17 on the two guard decisions (Flow 06)
REQ18	The protocol shall replace checkin_interval and grace_period on update_plan and reject zero or out-of-bounds values.	update_plan	UC5	REQ18 on the bounds decision and UPDATE Plan PDA box (Flow 06)
REQ19	The protocol shall reset plan.last_checkin to the Clock time on update_plan.	update_plan	UC5	REQ19 on the UPDATE Plan PDA box (Flow 06)
REQ20	The protocol shall overwrite plan.sweep_tx with a freshly signed partial sweep on update_plan.	update_plan	UC5	REQ20 on the sweep_tx size decision and UPDATE Plan PDA box (Flow 06)
REQ21	Client-side: the owner shall create a durable nonce account via the System Program with the keeper as authority, and record its pubkey in plan.nonce_account.	none (client, pre-step of UC1)	UC1	REQ21 on the "Create durable nonce account" step (Flow 02, Diagrams 01, 09)
REQ22	Client-side: the owner app shall build and partially sign [AdvanceNonceAccount, heir_sweep] with the keeper signature withheld (keeper = fee payer and nonce authority).	none (client; arg of UC1/UC2/UC5)	UC1, UC2, UC5	REQ22 on the "Build + partially sign" step (Flows 02, 03, 06, Diagram 09)
REQ23	Off-chain: the keeper shall scan all Plan PDAs daily and detect which are due.	none (keeper, off-chain)	UC3	REQ23 on the "Fetch all Plan PDAs" step (Flow 07, Diagram 09)
REQ24	Off-chain: the keeper shall simulate the sweep against the stored sweep_tx before signing, and broadcast the completed transaction only if the simulation passes.	none (keeper, off-chain)	UC3	REQ24 on the pre-flight and broadcast steps (Flow 07; "3 broadcast" arrows in Flow 04, Diagram 09)
REQ25	The Plan PDA shall hold exactly: owner, beneficiary, keeper, nonce_account, checkin_interval, grace_period, last_checkin, status, sweep_tx, bump.	state layout (all handlers)	UC1-UC5	REQ25 on the Plan PDA boxes (Diagrams 01, 08, 09, Flow 02)

Req	Atomic statement	Anchor handler	UC	Diagram label
REQ26	Off-chain hard guard: the keeper shall refuse to sign unless, at the moment of signing, the Plan freshly re-read from the chain at finalized commitment shows $\text{Clock} \geq \text{last_checkin} + \text{checkin_interval} + \text{grace_period} + \text{a safety margin}$.	none (keeper, off-chain)	UC3	REQ26 on the “HARD GUARD” decision (Flow 07), the keeper box in Flow 04 and Diagram 09

3. Granularity self-check (team pass, before any AI)

Rule	UC1 initialize_plan	UC2 check_in	UC3 heir_sweep	UC4 close_plan	UC5 update_plan
1. Atomicity (exactly one handler)	Pass. Creates one PDA, including the first sweep_tx. Nonce account creation is a separate System Program step done beforehand, not part of this handler.	Pass. One write to the Plan: last_checkin and replacement sweep_tx. Building and signing the bytes is client-side.	Pass with caveat. The transaction also contains AdvanceNonceAccount (System Program). That is a required companion instruction, not a second Heirloom state transition.	Pass. One transition: close the Plan PDA.	Pass. One transition on the Plan: schedule, last_checkin and sweep_tx replaced.
2. State ownership	Pass. All state in the program-owned Plan PDA.	Pass. Plan PDA.	Pass. Status in the Plan PDA; lamports move between System-owned wallets via CPI.	Pass. Plan PDA closed; rent returned.	Pass. Plan PDA.
3. Real signers	Pass. Owner signs.	Pass. Owner signs.	Pass with caveat. Owner signed weeks earlier offline; keeper signs at execution. The keeper is a named, on-record signer whose key is stored in plan.keeper, not a hidden backend.	Pass. Owner only, on purpose, so cancelling never depends on keeper liveness.	Pass. Owner signs.
4. On-chain vs client	On-chain: PDA creation and validation. Client: nonce account creation, building the partial sweep, UI.	On-chain: timestamp and sweep_tx write. Client: reminders, building and signing the fresh partial sweep.	On-chain: deadline gate, bindings, transfer, status. Off-chain: deadline polling, pre-flight simulation, final signature, broadcast.	On-chain: close and rent return. Off-chain: keeper nonce burn (REQ16).	On-chain: schedule and sweep_tx write. Client: building the replacement partial sweep.

Open issues found in our own pass

1. The half-signed transaction is stored on-chain in the Plan PDA, unencrypted. The withheld keeper signature makes the bytes useless to anyone but the keeper. Trade-offs: the PDA needs space for a transaction (max 1,232 bytes) and the bytes are public. Neither blocks the MVP.

2. The keeper is a single point of liveness and its key has protocol-wide blast radius. HSM / KMS and an N-of-M signer set are post-MVP. Until then the REQ26 guard is the control that stops the keeper from signing early.
3. Fixed amount with max failover is not in the MVP; the sweep takes the live balance only.
4. Because the keeper is the nonce authority it could advance or close the nonce and withhold (DoS), but it **cannot redirect funds**: the beneficiary is inside the owner's signed message and is re-checked against plan.beneficiary.
5. update_plan deliberately overlaps check_in (both reset last_checkin and overwrite sweep_tx). It stays separate because it also changes interval and grace, which check_in must never do.

4. Adversarial analysis (AI red-team, second pass)

The AI was used only as a critic of the use cases above, never to generate them. Findings are recorded as: caught that we missed; flagged but we judged wrong; accepted as an MVP limit.

#	AI finding	Rule	Outcome	Reasoning
A1	heir_sweep expects AdvanceNonceAccount as instruction 0; if the program does not check, a caller could omit it and the owner's signature is consumed in a non-durable transaction.	Atomicity / real signers	Caught, we missed it. Accepted. Added REQ11.	Without a nonce binding the signature would only be valid against a recent blockhash, so the real window is tiny, but an instructions-sysvar check is cheap and removes ambiguity.
A3	Keeper holds the nonce authority, so it can close or advance the nonce at will.	Real signers	Valid, partly accepted. Documented as a trust assumption.	A withholding / DoS risk, not theft: the beneficiary is fixed in the owner's signed message and re-checked on-chain (REQ09).
A4	check_in does not invalidate the old presigned transaction on-chain; invalidation relies on the keeper advancing the nonce.	On-chain vs client	Partly valid. Documented.	check_in does not touch the nonce, so an old half-signed copy survives. It is harmless: it lacks the keeper signature and fails the deadline gate (REQ08) against the new last_checkin. sweep_tx is overwritten on every check-in (REQ07).
A2	Sweeping the owner to 0 lamports may leave a non-rent-exempt account or fail.	State ownership	Flagged, we judged it wrong.	System-owned wallets can be fully drained; there is no rent requirement at zero balance. It matters only if the wallet holds token accounts or other rent-bearing state. Fees are paid by the keeper, so the owner balance is not needed for fees.
A5	A compromised owner key near the trigger could drain funds first.	Real signers	Out of scope.	Owner key compromise defeats any non-custodial scheme; disclosed in the LOI.
A6	Seeding the Plan PDA only by owner allows one plan per owner.	State ownership	Accepted as an MVP limit.	Multi-beneficiary / multi-plan is roadmap.
A7	"UC3 contains two instructions", which violates 1 use case = 1 handler.	Atomicity	Flagged, we judged it wrong. Rejected.	AdvanceNonceAccount is a required System Program companion for any durable-nonce transaction, not an Heirloom state transition. Only heir_sweep is ours.

#	AI finding	Rule	Outcome	Reasoning
A8	Storing a signed transaction publicly on-chain leaks it; it should be encrypted.	State ownership / real signers	Flagged, we judged it wrong. Rejected.	The transaction lacks the keeper's required signature, so nobody else can submit it or burn the nonce. Encryption would add cost and a key to manage for no gain. Residual: it reveals the beneficiary pubkey, which the plan already exposes.

5. On-chain requirements matrix

5.1 Accounts and PDAs

Account	Type	Owner program	Authority	Seeds / address	Mutated by	Holds
Plan	PDA	Heirloom	plan.owner (has_one)	["plan", owner_pubkey] + stored bump	UC2, UC3, UC5; closed by UC4	owner, beneficiary, keeper, nonce_account, checkin_interval, grace_period, last_checkin, status, sweep_tx ($\leq 1,232$ B), bump (REQ25)
Nonce account	Keypair account	System Program	keeper (constant)	Keypair, pubkey in plan.nonce_account	Advanced in UC3 ix 0; withdrawn by keeper (REQ16)	Durable nonce value + rent-exempt lamports
Owner wallet	Wallet (keypair)	System Program	owner	Owner pubkey	Debited in UC3	The assets (never moved before the trigger)
Beneficiary wallet	Wallet	System Program	beneficiary (signs nothing)	Beneficiary pubkey	Credited in UC3	—
Keeper wallet	Wallet (keypair)	System Program	keeper	Keeper pubkey	Pays fees	—
Sysvars	Runtime	Runtime	—	Clock, Instructions, RecentBlockhashes	Read-only	—

No authority transfers happen in the MVP. Authority per decision point: Plan is controlled by the owner for UC1, UC2, UC4, UC5; UC3 writes only status, by program logic. The nonce is always under the keeper. The owner wallet's authority is exercised in UC3 through the presigned signature.

5.2 Instruction handlers

Handler	Signers	Accounts	Key validations
<code>initialize_plan(beneficiary, keeper, nonce_account, interval, grace, sweep_tx)</code>	owner	owner, plan (init), nonce_account, system_program	interval/grace > 0 and in bounds (REQ02); nonce authority == keeper (REQ03); sweep_tx within allocated size (REQ04); plan must not exist (REQ01)
<code>check_in(sweep_tx)</code>	owner	owner, plan (mut)	has_one = owner; status Active (REQ05); sweep_tx within size (REQ07)
<code>heir_sweep()</code>	owner (presigned), keeper	owner (mut), beneficiary (mut), plan (mut), nonce_account, clock, instructions sysvar, system_program	REQ08 to REQ12
<code>close_plan()</code>	owner	owner (mut), plan (mut, close = owner)	has_one = owner; status Active or Distributed (REQ15)
<code>update_plan(interval, grace, sweep_tx)</code>	owner	owner, plan (mut)	has_one = owner; status Active (REQ17); bounds (REQ18); sweep_tx within size (REQ20)

5.3 CPI dependencies

- **System Program transfer** from `heir_sweep` (REQ13), authorised by the owner's presigned signature, not a PDA signer.
- **System Program AdvanceNonceAccount**: top-level companion instruction in the same transaction (instruction 0), not a CPI from our program (REQ11).
- `close_plan` uses Anchor's `close` constraint (lamport transfer, no CPI).
- Off-chain keeper transactions: `AdvanceNonceAccount` then `WithdrawNonceAccount` (REQ16).
- No Token Program CPI in the MVP.

5.4 Custom program constraints (error codes)

Error	Raised when	Req
<code>InvalidSchedule</code>	interval or grace is zero or out of bounds	REQ02, REQ18
<code>BadNonceAuthority</code>	nonce account authority is not the keeper	REQ03
<code>SweepTxTooLarge</code>	sweep_tx exceeds the allocated space	REQ04, REQ07, REQ20
<code>AccountAlreadyInUse</code>	a Plan PDA already exists for this owner	REQ01
<code>ConstraintHasOne</code>	signer is not plan.owner	REQ05, REQ15, REQ17
<code>PlanNotActive</code>	status is not Active for <code>check_in</code> / <code>update_plan</code>	REQ05, REQ17
<code>InvalidStatus</code>	<code>close_plan</code> on an unexpected status	REQ15
<code>MissingNonceAdvance</code>	instruction 0 is not <code>AdvanceNonceAccount</code> on <code>plan.nonce_account</code>	REQ11

Error	Raised when	Req
DeadlineNotReached	Clock < last_checkin + interval + grace	REQ08
BeneficiaryMismatch	beneficiary or owner account differs from the plan	REQ09
KeeperOrNonceMismatch	keeper signer or nonce account differs from the plan	REQ10
AlreadyDistributed	status is not Active in heir_sweep	REQ12

5.5 External dependencies

System Program (nonce creation, advance, withdraw, transfer); Clock, Instructions and RecentBlockhashes sysvars; Solana RPC for the keeper; keeper infrastructure and signing key (HSM / KMS planned post-MVP); owner wallet app (client). The half-signed transaction lives in the Plan PDA, so there is no separate storage service.

5.6 Alternate flows and failure paths

Case	Where it shows	Outcome
Early submission by someone holding the owner-signed bytes	Flow 04, signature decision	Rejected at signature verification; nonce not advanced
Keeper submits before the deadline (bug or compromised key)	Flow 04, REQ08	Reverts and the nonce is advanced (the POC fragility). The on-chain gate (REQ08) means no funds can move, but the nonce is lost. Prevented by the keeper hard guard (REQ26) plus pre-flight (REQ24). Residual risk is keeper key compromise (trust assumption), mitigated post-MVP by HSM / KMS and an N-of-M signer set.
Owner checks in	Flow 03	last_checkin updated, sweep_tx overwritten; the old copy is obsolete
Second sweep attempt	Flow 04, REQ12	AlreadyDistributed
Wrong beneficiary / nonce / keeper	Flow 04, REQ09, REQ10	Revert
Owner cancels the plan	Flow 05	Plan closed, sweep_tx deleted; keeper burns the nonce (REQ16); stale bytes are dead. Until then they are inert (no plan, no keeper signature)
Keeper offline when the owner cancels	Flow 05	Cancel still succeeds; nonce burn delayed; re-initialising needs a new nonce account anyway
Owner changes interval / grace	Flow 06	Timer restarts from now; new partial sweep replaces the old one
update_plan after Distributed	Flow 06	Rejected (PlanNotActive); close_plan is still allowed to reclaim rent

6. Architecture diagrams

Structure mirrors the reference: one overview, one flow per instruction, the keeper flow, the account map, then one end-to-end diagram showing every external actor. Colours distinguish programs, accounts, signers,

beneficiaries and external services (legend on each page). Signers have a thick border; the beneficiary has a dashed border.

- Figure 01: Overview, program structure
- Figures 02 to 06: one flow per instruction (initialize_plan, check_in, heir_sweep, close_plan, update_plan)
- Figure 07: Keeper off-chain flow
- Figure 08: Accounts, PDAs and authorities
- Figure 09: End-to-end architecture

Figure 01: Overview, program structure

Numbered actor-to-instruction arrows. Signers are thick-bordered; the beneficiary is dashed.

Diagram 01: Overview, program structure and instruction arrows

Every arrow between an actor and the program is numbered and maps to one instruction (1 UC = 1 state transition = 1 handler).

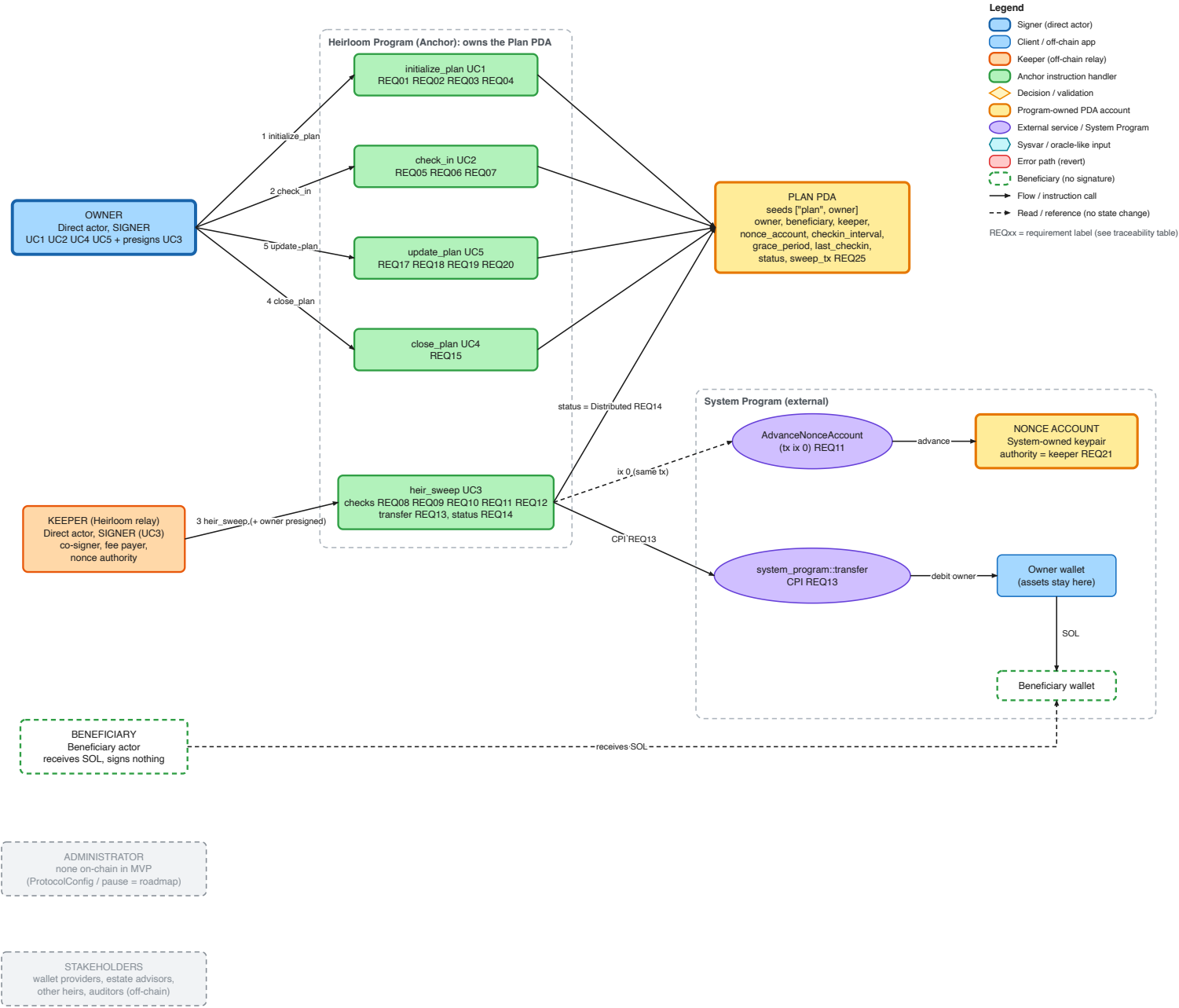


Figure 02: initialize_plan (UC1)

REQ01 to REQ04, REQ21, REQ22, REQ25.

Flow 02: initialize_plan (UC1)

Owner creates the succession plan. REQ01 to REQ04, with client-side steps REQ21 and REQ22.

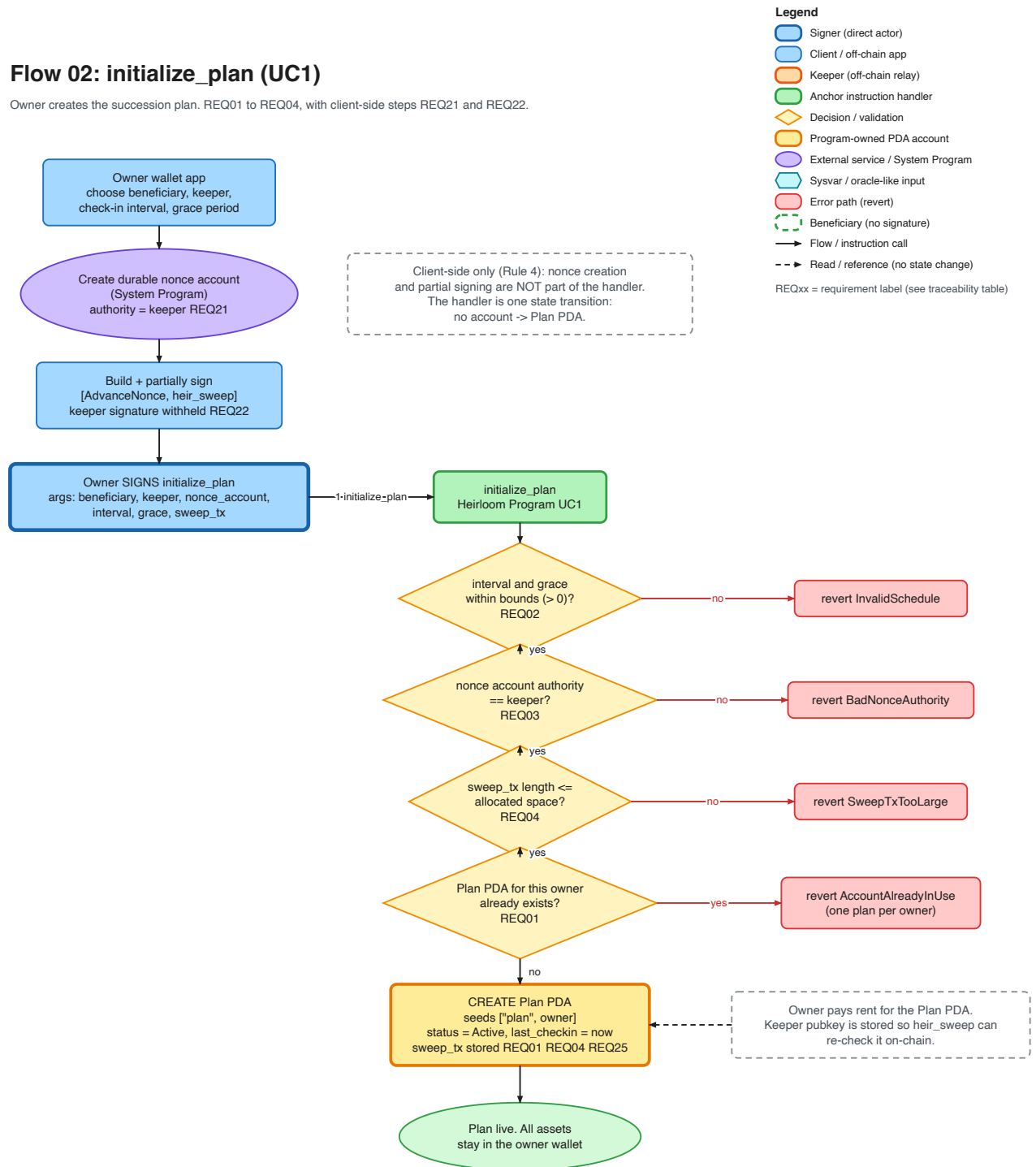


Figure 03: check_in (UC2)

REQ05 to REQ07, REQ22.

Flow 03: check_in (UC2)

Owner proves liveness every cycle. REQ05 to REQ07 (REQ22 client-side).

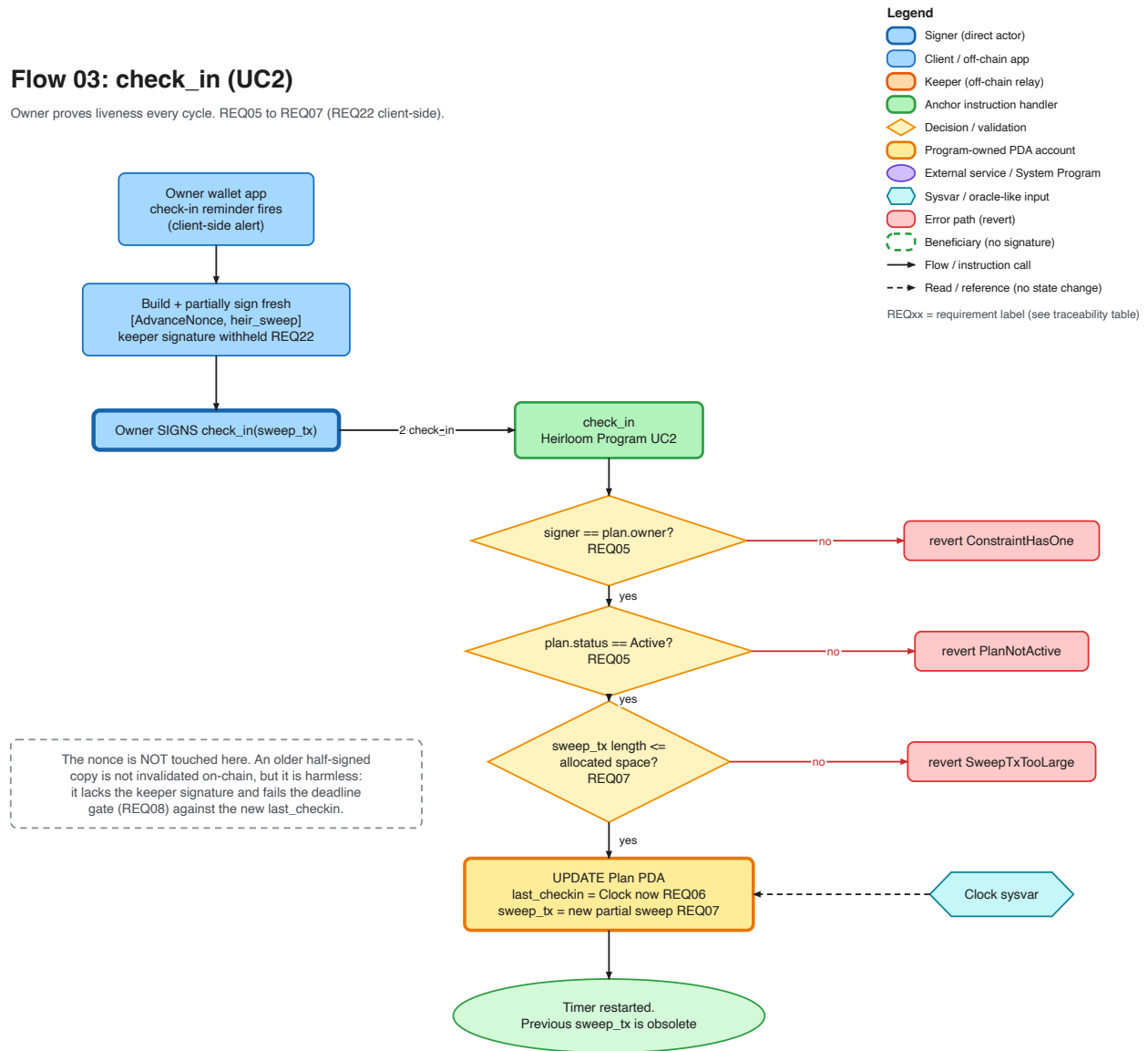


Figure 04: heir_sweep (UC3)

REQ08 to REQ14, REQ24, REQ26.

Flow 04: heir_sweep (UC3), the on-chain transaction

Keeper completes the owner-presigned partial transaction after the deadline. REQ08 to REQ14.

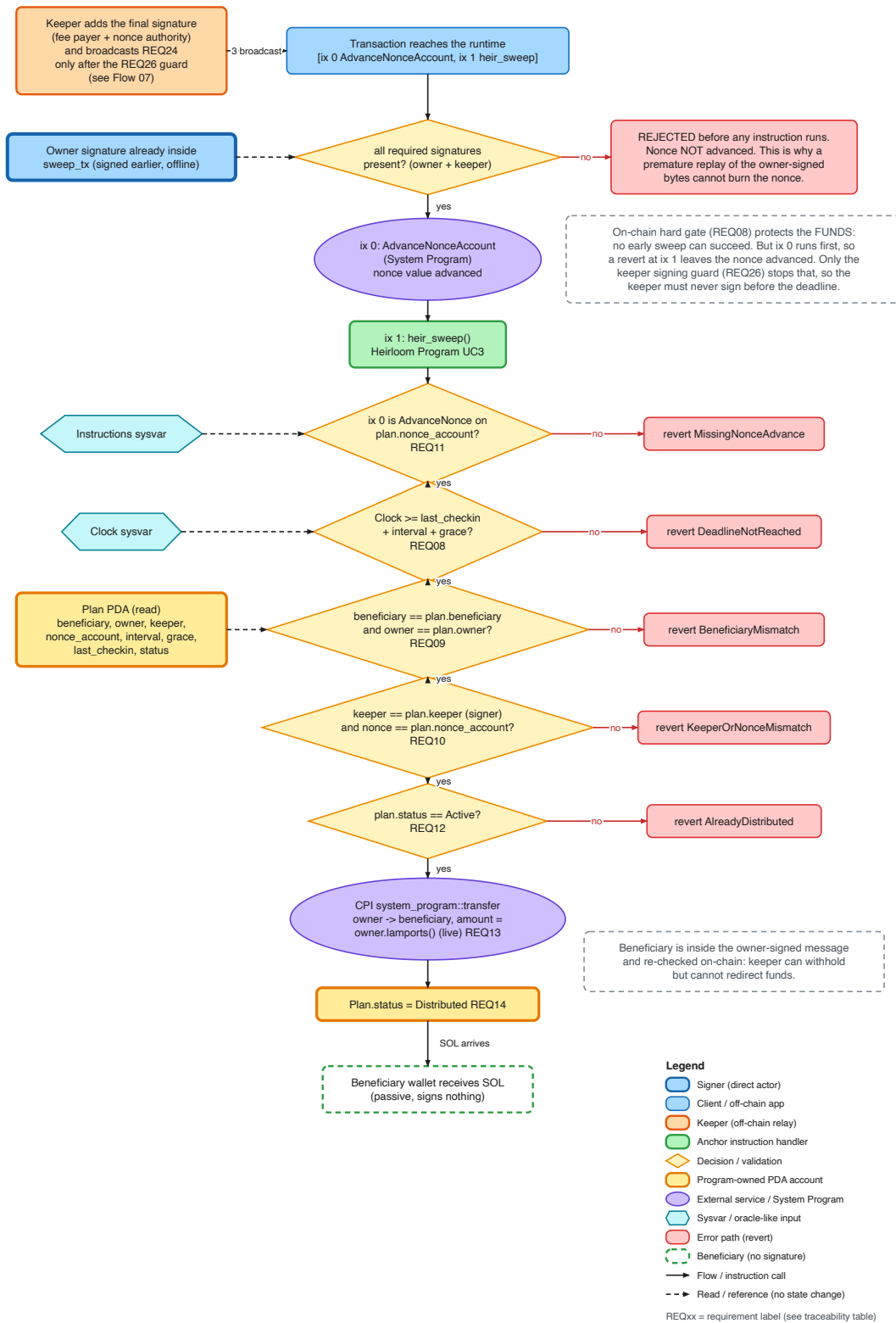


Figure 05: close_plan (UC4) and keeper nonce burn

REQ15, REQ16.

Flow 05: close_plan (UC4) and keeper nonce burn

Owner cancels the plan and removes the heir. REQ15 on-chain, REQ16 off-chain follow-up.

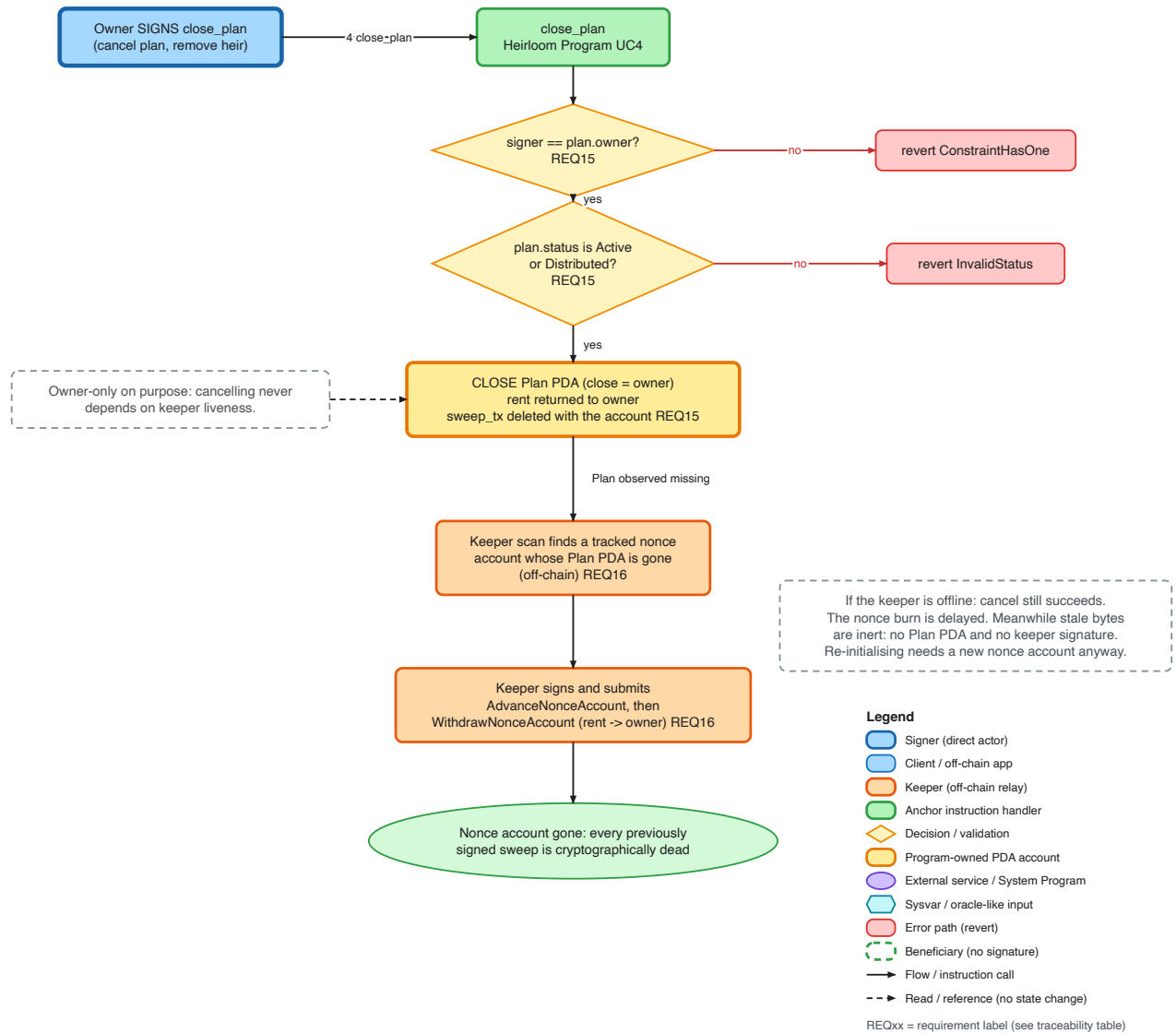


Figure 06: update_plan (UC5)

REQ17 to REQ20, REQ22.

Flow 06: update_plan (UC5)

Owner changes the check-in period; timer resets. REQ17 to REQ20 (REQ22 client-side).

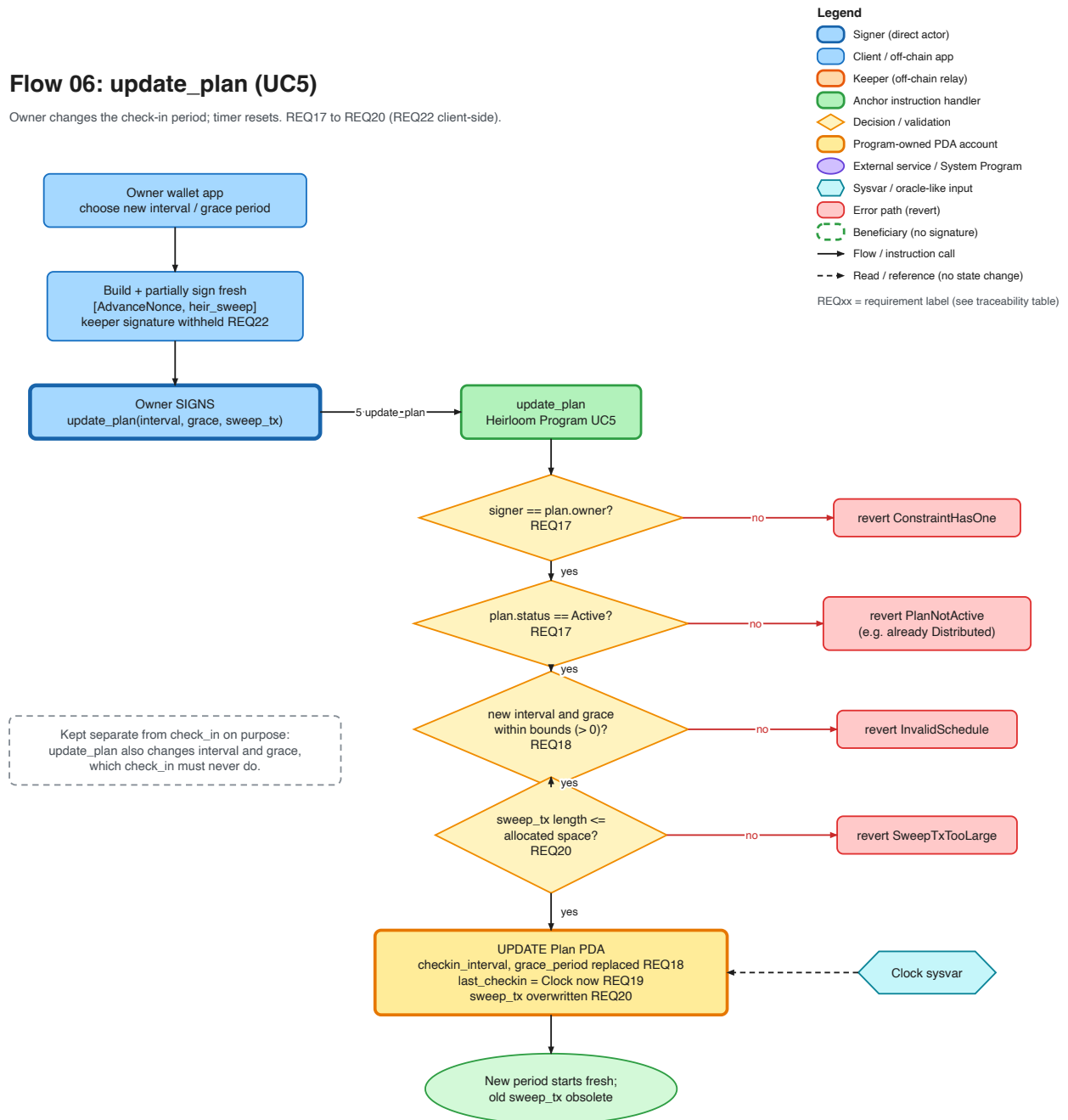


Figure 07: Keeper off-chain flow

REQ16, REQ23, REQ24, REQ26.

Flow 07: Keeper off-chain flow

Daily scan, pre-flight, completion of the sweep, and nonce burn. REQ16, REQ23, REQ24.

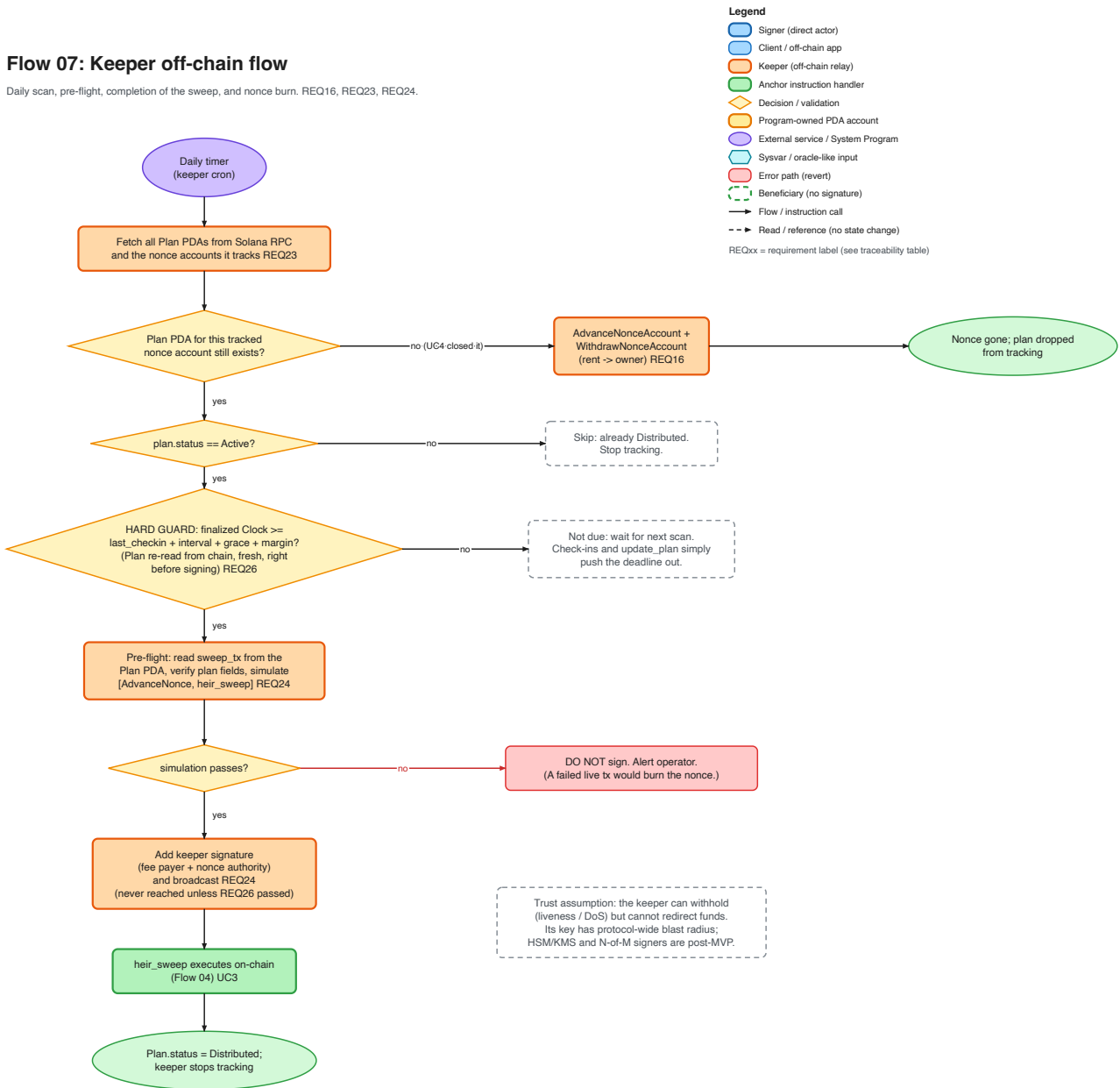


Figure 08: Accounts, PDAs and authorities

REQ25, REQ21, REQ13, REQ16, REQ22.

Diagram 08: Account structure, PDAs and authorities

Every on-chain account the MVP touches: owner program, authority, data, type.

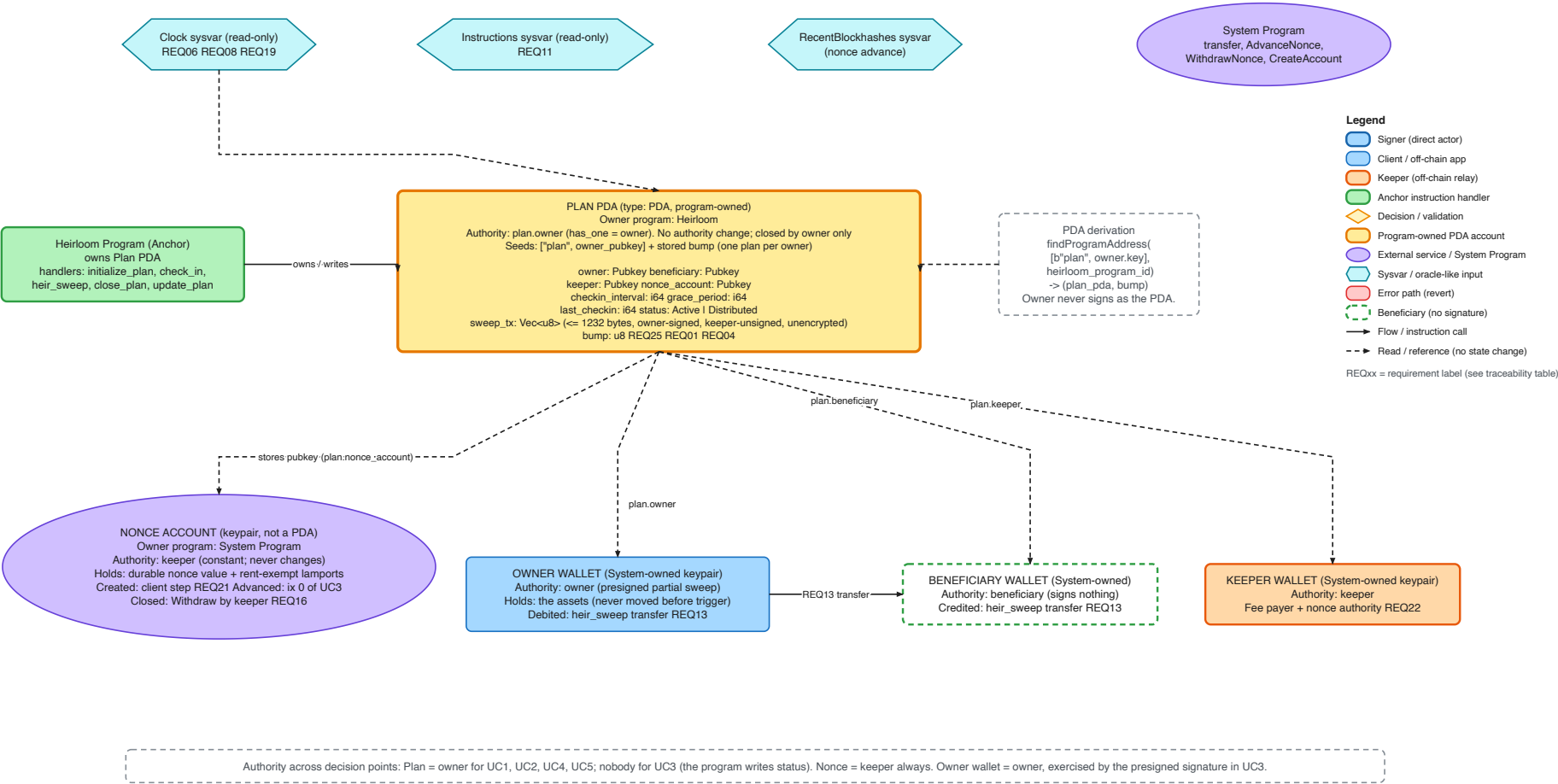
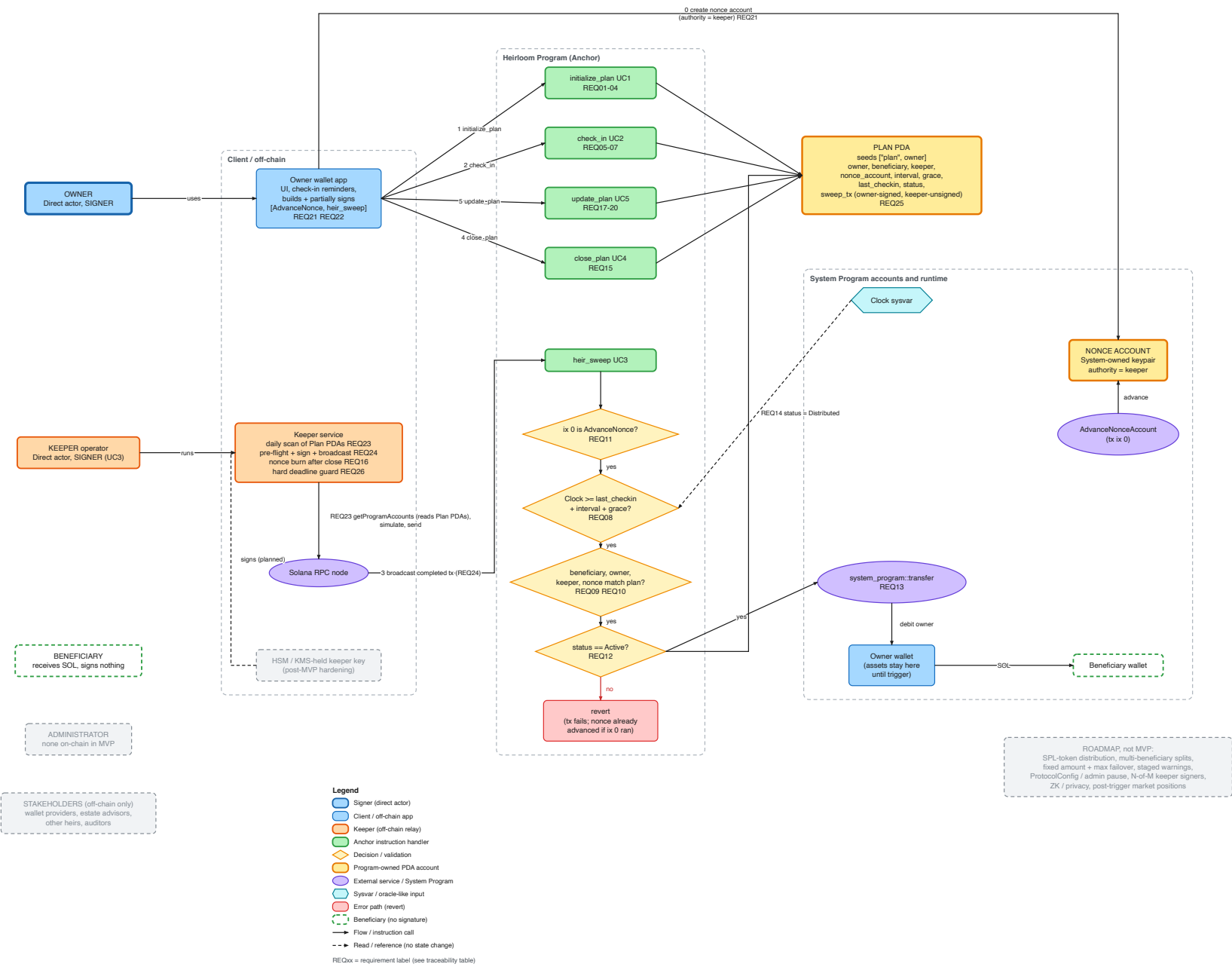


Figure 09: End-to-end architecture (entire protocol)

All actors, programs, accounts, external dependencies, validation points, error paths, and roadmap context.

Diagram 09: End-to-end architecture (entire protocol)

All actors, programs, accounts, external dependencies, validation points, error paths and roadmap context. Labels REQxx sit next to the flow that implements them.



7. Use-case to diagram traceability

Use case	Handler	Where on the diagrams	Requirements
UC1 Create plan	<code>initialize_plan</code>	Figure 02; arrow “1” in Figures 01, 09; Plan PDA in Figure 08	REQ01–04, REQ21, REQ22, REQ25
UC2 Check in	<code>check_in</code>	Figure 03; arrow “2” in Figures 01, 09	REQ05–07, REQ22
UC3 Sweep	<code>heir_sweep</code>	Figure 04; arrow “3” in Figures 01, 09; Figure 07 (keeper side)	REQ08–14, REQ23, REQ24, REQ26
UC4 Cancel plan	<code>close_plan</code>	Figure 05; arrow “4” in Figures 01, 09; Figure 07 (nonce burn)	REQ15, REQ16
UC5 Update period	<code>update_plan</code>	Figure 06; arrow “5” in Figures 01, 09	REQ17–20, REQ22

Final checklist

- All programs represented: Heirloom Program and System Program (Figures 01, 09).
- Account structures mapped: Figure 08, with owner, authority, data and type for every account.
- PDA seeds shown: `["plan", owner]` with derivation (Figure 08).
- Program interactions and CPIs illustrated: Figures 01, 04, 09.
- Validation points and error paths: decision diamonds and red revert boxes in Figures 02 to 07, 09.
- External dependencies shown: System Program, RPC, sysvars, keeper, HSM / KMS (planned), roadmap items.
- All actor categories represented, signers visually distinguished from beneficiaries: Figures 01, 09.
- Every item of the on-chain requirements matrix appears on a diagram, labelled REQxx.

Sources

Deliverable 1 LOI and submission, execution-mechanism notes, the heir-sweep POC. Prior art: Dead Man's Vault, Eternal Key, Deadhand Protocol.